Project Title	Rising Waters: Machine Learning Based Flood Prediction System		
Developer	N. Joshnavi devi	Team ID	[Team ID]
Date	28 to 29 June	Phase / Document	Project Development Phase

13. Coding & Solution

This section walks through the two pipelines that make up the system's actual logic, in the same order they run: first the offline training pipeline that produces the serialized model and scaler, then the online prediction pipeline that consumes them on every request.

13.1 Training Pipeline (train.py)

- Load flood_prediction.csv into a Pandas DataFrame.
- Rename columns (Temp, Humidity, Cloud Cover, ANNUAL, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, avgjune, sub, flood) to the application's field naming convention.
- Impute missing values in feature columns using the column median; drop any remaining nulls.
- Split features (X) and target (Flood_Risk / y), then perform an 80/20 stratified train-test split (random_state=42).
- Fit a StandardScaler on the training features and transform both train and test sets.
- Train four classifiers — Decision Tree (max_depth=15), Random Forest (100 estimators, max_depth=15), K-Nearest Neighbors (k=5), and XGBoost (100 estimators, logloss) — and compare test-set accuracy.
- Select the highest-accuracy model, print its classification report and confusion matrix, then serialize both the model and the scaler with Joblib into models/.

The column-renaming step deserves particular attention: it is the single point in the codebase where the raw dataset's headers are translated into the field names the Flask form actually submits, which means this mapping must be updated in exactly one place if the dataset schema ever changes.

13.2 Prediction Pipeline (app.py)

- On startup, verify that flood_model.joblib and scaler.joblib exist, then load them.
- The /predict route reads 10 form fields (temp, humidity, cloud_cover, annual_rainfall, jan_feb, mar_may, jun_sep, oct_dec, avgjune, sub) and casts them to float.
- Inputs are assembled into a single-row DataFrame, transformed with the loaded scaler, and passed to the model's predict() and predict_proba().
- The probability of the positive (flood) class is converted to a percentage and classified as: ≥ 75% → Severe Flood Warning, ≥ 40% → Elevated Flood Watch, otherwise → Normal Conditions.
- Threshold-based rules (e.g. annual_rainfall > 2500 mm, jun_sep > 1500 mm, humidity > 80%, cloud_cover > 70%) generate contextual insight messages and matching safety protocols, which are rendered back into index.html.

Worth noting is that these insight thresholds are only evaluated when the severity tier is already “high” or “medium”; for Normal Conditions, a separate, reassurance-oriented set of messages is generated instead (e.g. noting that rainfall or humidity levels are within expected historical norms), so the tone of the explanation always matches the tone of the result.